
Desktop GUI and Cross-Platform Streaming

Snapchat Masking and Hacking

Zoe J.D.M. Spyrou
Student Number: 452280
Fontys HBO-ICT
Semester 4 OL

Table of Contents

- [Snapchat Masking and Hacking](#)
- [Context](#)
- [Objective](#)
- [Scope](#)
- [Research Methodology](#)
 - [Process](#)
 - [Initial Desktop Streaming Attempts](#)
 - [Selection of scrcpy as Streaming Backbone](#)
 - [Custom Python-Based Decoder and GUI](#)
 - [Media Conversion Pipeline](#)
 - [Building the GUI into a single application](#)
 - [Reasoning Against Docker](#)
 - [Integration With the Virtual Camera Module](#)
 - [Testing and Validation](#)
 - [Mermaid Diagram of GUI Simplified](#)
 - [Next Steps](#)
 - [Risks](#)
 - [Sources](#)

Date	Task
08/01/2026	1st Version

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snapscore not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

This document continues on the previous document, [Replacing the Android Phone's Camera With a Virtual Camera](#), as well as [Translating the solution to a user-friendly end product](#). In there, the Android camera stream pipeline is intercepted with the use of an Xposed/LSPosed module. While this solution successfully replaces the camera feed at the framework level, it requires manual interaction with the Android file system: converting footage, placing it in a specific directory, and validating the output directly on the device. The basic reasoning behind the GUI's choice is also explained in [Translating the solution to a user-friendly end product](#).

To improve usability and operational speed, a desktop-side control and streaming system was developed. This system allows investigators to stream, preview, and validate the virtual camera output in real time from a single desktop interface. The desktop component does not replace the Android module. Instead, it acts as a control, validation, and interaction layer on top of the existing camera interception.

By separating responsibilities between the Android module (camera interception) and the desktop application (control and validation), the overall system becomes more reliable, easier to operate, and better suited for investigative environments where time pressure and operational clarity are critical.

Objective

The primary objective of this project is to send images and videos through Snapchat in such a way that they appear to be freshly captured by the device's camera, without triggering Snapchat's "media upload" tag.

A secondary objective is to design and implement a low-latency desktop streaming service that allows investigators to:

- Preview exactly what the target application receives
- Interact with the device through touch input
- Reduce the number of physical devices that must be handled during an investigation

Scope

- A desktop-based GUI for previewing and validating the Android camera output
- A streaming pipeline for real-time device screen capture
- Touch-event forwarding from desktop to mobile device
- Integration with existing open-source components such as scrcpy, FFmpeg, and PyAV

The Android camera interception module itself is considered an external dependency and is documented separately.

Research Methodology

The project follows the DOT framework and applies the following methods:

Method	Application
Available Product Analysis	Evaluation of existing mirroring tools
Multi-Criteria Decision Making	Comparison of streaming and decoding approaches based on latency and cross-platform compatibility
Literature Study	Review of Android camera APIs, HAL behavior, and media pipelines
Stakeholder Analysis	Regular contact sessions with stakeholder Ronald Beljaars

Process

Initial Desktop Streaming Attempts

The first step toward a desktop-side solution was to investigate whether the Android camera output could be accessed directly through existing desktop tools. Early attempts focused on using raw TCP streams, or adb's screenrecorder to receive the video feed from the device. While a connection could be established, the stream lacked framing information and caused frequent freezes.

These experiments demonstrated that Android camera output, especially when routed through nonstandard pipelines, cannot be treated as a generic video stream without careful handling of codec boundaries, buffering, and synchronization.

The screenrecorder functions that were tested in the first few attempts was built on the following adb command and edits of it (Jason.Chuang, 2016),

```
adb exec-out screenrecord --output-format=h264 --size
360x480 --bit-rate 128000 - | ffmpeg -f h264 -probesize 32 -
framedrop -flags low_delay -analyzeduration 0 -
```

Although this method can produce a valid video stream when written to a file, it proved unreliable for real-time streaming. The output is optimized for recording rather than live playback, and small delays in the data stream caused FFmpeg to stall or lose synchronization.

The ADB server initialization was implemented as follows:

```
def start_adb_server():
    adb_path = shutil.which("adb")
    if adb_path:
        try:
            subprocess.run([adb_path, "start-server"],
check=False)
        except Exception as e:
            print(f"Failed to start ADB server: {e}")
```

```
else:  
    print("Please install ADB")
```

This approach ensures the ADB server is running before attempting any device communication, with proper error handling for environments where ADB may not be installed, which should not be an actual concern though, as this is an issue that will be prevented once the app is built and all dependencies are internally managed.

A second approach relied on manually forwarding the scrcpy server over a TCP socket (User, 2012) and connecting to it from a custom Python application:

```
`adb forward tcp:1234 localabstract:scrcpy && adb shell  
CLASSPATH=/data/local/tmp/scrcpy.jar app_process /  
com.genymobile.scrcpy.Server 3.3.4 tunnel_forward=true  
control=false cleanup=false max_size=1920 audio=false  
video_codec_options='profile=1,level=4096' &`
```

On the desktop side, a Python prototype connected to the socket and forwarded the received H.264 byte stream directly into an FFmpeg subprocess for decoding. While this worked in principle, several became apparent:

- The stream contained no resolution metadata, requiring the decoder to assume frame size from FFmpeg log output, making it impossible to log correct coordinates for touch events
- Incorrect buffering caused blocking reads, resulting in frozen video
- Small timing variations could desynchronize the decoder and stall the pipeline entirely
- Multi-threaded handling was required just to keep the stream responsive

These limitations made it clear that Android video output cannot be treated as a generic video feed. Without a defined protocol layer, explicit framing, and controlled buffering, the stream remains fragile and unsuitable for operational use.

Moreover, integrating touch support into these attempts, proved futile as well. The screen size, while correctly read and scaled, could not be reliably mapped to the video frames delivered by the pipeline. Even minor delays in frame arrival or dropped frames caused a mismatch between the reported coordinates and the visual output. This made interactive control nearly impossible without additional synchronization mechanisms.

An example of the severity: when pressing q on the phone's keyboard, it registered the coordinates of the letter e, w=r, e=r, r=t, t=t, y=y, u=y, i=u, o=i, p=i etc.

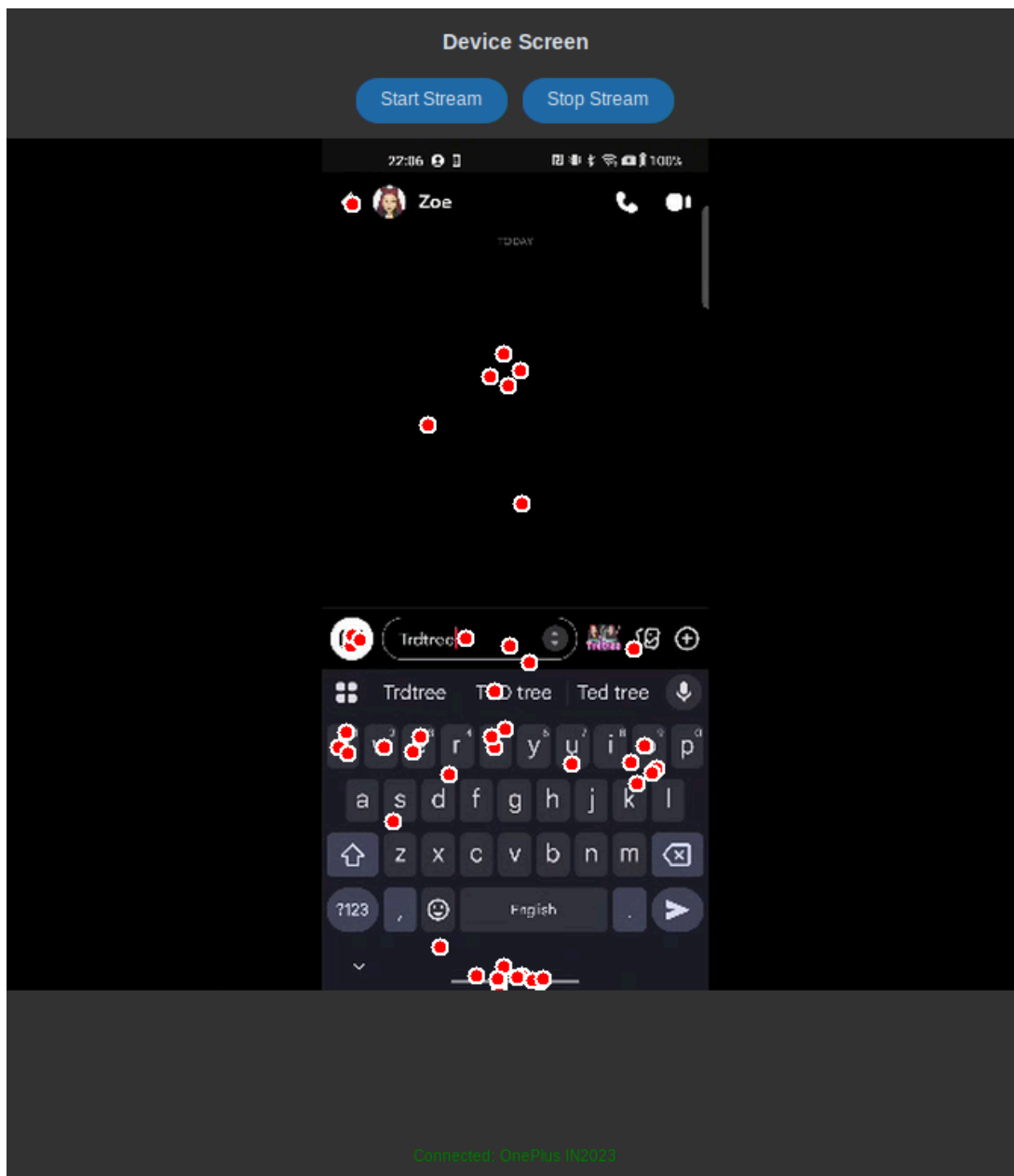


image-106.png

Selection of scrcpy as Streaming Backbone

After evaluating several tools, scrcpy was selected as the foundation for the desktop streaming component. It still could not be used "as-is", as was also described in: [Translating the solution to a user-friendly end product](#), due to scrcpy being designed as a standalone tool and relying on X11 on Linux, which makes embedding it into a cross-platform Python GUI difficult. On Windows, scrcpy runs in a separate window and does not expose a straightforward way to embed its output, and containerization solutions such as Docker do not resolve this limitation.

The decision was based on the following criteria:

- scrcpy is open source and free to use, aligning with cost-conscious software selection
- It provides low-latency H.264 streaming over ADB
- It supports both video streaming and input forwarding
- It is actively maintained and widely used

Aspect	Initial Desktop Streaming Attempts	scrcpy as Streaming Backbone	Custom Python-Based Decoder & GUI
Method	Direct TCP streams or adb screenrecord output	Use scrcpy as-is (modified embedding considered)	Python client connects to scrcpy server over TCP, decodes H.264 in Python
Implementation Complexity	Low to medium for simple streaming, high for stable real-time playback	Medium: scrcpy handles streaming but not easily embeddable	High: requires custom decoder, GUI, and input forwarding
Video Stream Handling	No framing or metadata: prone to freezes and desynchronization	Properly handles H.264 streaming with low latency	Explicit decoding with PyAV: full control over frame handling
Input Handling	Not handled: limited to video only	Supports input forwarding	Full mouse-to-touch mapping via ADB
Latency	High and unstable	Low	Low: with controlled buffering
Stability & Reliability	Unstable: frequent stalls and freezes	Stable	Stable: fully controlled pipeline
Cross-Platform Integration	Difficult: no standard API for embedding	Difficult: scrcpy window cannot easily embed into GUI + different setups across Windows and Linux	Easy: fully Python-based GUI, cross-platform capable

Aspect	Initial Desktop Streaming Attempts	scrcpy as Streaming Backbone	Custom Python-Based Decoder & GUI
Pros	Simple to prototype: uses built-in Android tools	Free, open-source, low-latency, widely used	Full control: fully embeddable, real-time validation, input support
Cons	Unreliable for real-time use: no frame metadata	Standalone window, embedding is tricky: not designed for custom GUI	High implementation effort: requires handling decoding, threading, GUI

Custom Python-Based Decoder and GUI

Rather than embedding the official scrcpy client, a custom Python-based decoder and GUI were implemented. This approach allowed full control over the decoding process and seamless integration with the investigative workflow.

The desktop application performs the following tasks:

- Establishes a TCP connection to the scrcpy server

```
def setup_adb_reverse(self):
    print("[*] Setting up adb reverse...")
    result = subprocess.run(
        ['adb', 'reverse', 'localabstract:scrcpy',
f'tcp:{self.port}'],
        capture_output=True,
        text=True
    )
    if result.returncode != 0:
        raise RuntimeError(f"Failed to set up adb
reverse: {result.stderr}")
    print("[+] ADB reverse configured")

def start_listener(self):
    print(f"[*] Starting listener on port
```

```
{self.port}...")
    self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    self.server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
    self.server_socket.bind(('127.0.0.1', self.port))
    self.server_socket.listen(1)
    print(f"[+] Listener started on 127.0.0.1:
{self.port}")
    ...
```

- Receives a raw H.264 byte stream

One of the challenges was parsing the raw H.264 stream without frame metadata. The solution involved detecting NAL (Network Abstraction Layer) unit boundaries and feeding them to the PyAV decoder

- Decodes frames using PyAV (PyAV-Org, n.d.) (Voce, 2022)

```
```python
def find_nal_units(self, data):
 nal_units = []
 i = 0
 start = -1

 while i < len(data) - 3:
 if data[i:i+4] == b'\x00\x00\x00\x01':
 if start >= 0:
 nal_units.append((start, i))
 start = i
 i += 4
 elif data[i:i+3] == b'\x00\x00\x01':
 if start >= 0:
 nal_units.append((start, i))
 start = i
 i += 3
 else:
 i += 1

 return nal_units, start

def decode_frames_thread(self, conn):
 print("[*] Starting H264 decoder with PyAV...")

 codec = av.CodecContext.create('h264', 'r')
 codec.options = {
```

```

 'flags': 'low_delay',
 'flags2': 'fast',
 }
 codec.skip_frame = 'NONREF'
 codec.thread_type = 'SLICE'

```

- Displays frames using OpenCV (*OpenCV: OpenCV Modules*, n.d.)  
The stream display is embedded in a Tkinter Canvas widget:

```

class ScrcpyCanvasWrapper:
 def __init__(self, canvas, port=27183,
max_size=1080, max_fps=24):
 self.canvas = canvas
 self.port = port
 self.max_size = max_size
 self.max_fps = max_fps

 self.stream = None
 self.running = False
 self.current_frame = None
 self.frame_lock = threading.Lock()

 self.canvas.bind("<Button-1>",
self._on_canvas_click)
 self.canvas.bind("<B1-Motion>",
self._on_canvas_drag)
 self.canvas.bind("<ButtonRelease-1>",
self._on_canvas_release)
 ...

- Forwards mouse input as touch events via ADB
```python
def map_to_device(self, frame_x, frame_y):
    if self.width is None or self.height is None:
        return None, None

    frame_w = self.width
    frame_h = self.height
    dev_w = self.device_screen_width if
self.device_screen_width else frame_w
    dev_h = self.device_screen_height if
self.device_screen_height else frame_h

    device_x = int(frame_x * dev_w / frame_w)

```

```

device_y = int(frame_y * dev_h / frame_h)

device_x = max(0, min(device_x, dev_w - 1))
device_y = max(0, min(device_y, dev_h - 1))

return device_x, device_y

```

This architecture enables real-time validation of the virtual camera feed and allows investigators to interact with the device without touching the phone itself.

Media Conversion Pipeline

Before media can be used as virtual camera input, images must be converted to video format at the correct resolution. This conversion is handled by FFmpeg:

```

def convert_image_to_video(image_path,
output_video_path, duration=10, target_width=1920,
target_height=1080):
    try:
        with tempfile.NamedTemporaryFile(suffix='.jpg',
delete=False) as temp_img:
            temp_image_path = temp_img.name

            if not resize_image_to_phone_camera(image_path,
temp_image_path, target_width, target_height):
                return False

            cmd = [
                'ffmpeg',
                '-loop', '1',
                '-i', temp_image_path,
                '-c:v', 'libx264',
                '-t', str(duration),
                '-pix_fmt', 'yuv420p',
                '-vf', 'fps=30',
                '-y',
                output_video_path
            ]

```

```

        result = subprocess.run(
            cmd,
            capture_output=True,
            text=True,
            timeout=30
        )

        if result.returncode != 0:
            print(f"FFmpeg error: {result.stderr}")
            return False

        return True
    except Exception as e:
        print(f"Error converting image to video: {e}")
        return False

```

Images are resized using Pillow with proper aspect ratio handling and cropping:

```

def resize_image_to_phone_camera(image_path,
    output_path, target_width=1920, target_height=1080):
    try:
        with Image.open(image_path) as img:
            if img.mode in ('RGBA', 'LA', 'P'):
                rgb_img = Image.new('RGB', img.size, (0,
0, 0))

                if img.mode == 'P':
                    img = img.convert('RGBA')
                    rgb_img.paste(img, mask=img.split()[-1])
            if img.mode in ('RGBA', 'LA') else None)
                img = rgb_img
            elif img.mode != 'RGB':
                img = img.convert('RGB')

            img_ratio = img.width / img.height
            target_ratio = target_width / target_height

            if img_ratio > target_ratio:
                new_height = target_height
                new_width = int(target_height *
img_ratio)
            else:
                new_width = target_width

```

```

        new_height = int(target_width /
img_ratio)

        img = img.resize((new_width, new_height),
Image.Resampling.LANCZOS)

        left = (new_width - target_width) // 2
        top = (new_height - target_height) // 2
        right = left + target_width
        bottom = top + target_height
        img = img.crop((left, top, right, bottom))

        img.save(output_path, 'JPEG', quality=95)
        return True
    except Exception as e:
        print(f"Error resizing image: {e}")
        return False

```

The device resolution is dynamically detected to ensure proper scaling:

```

def get_device_resolution(device):
    try:
        output = device.shell("wm size")
        size_line = output.strip()
        if "Physical size:" in size_line:
            resolution = size_line.split("Physical
size:")[1].strip()
        else:
            resolution = size_line.split(":")
[-1].strip()

        width, height = map(int, resolution.split('x'))
        return (width, height)
    except Exception as e:
        print(f"Error getting device resolution: {e}")
        return (1080, 1920)

```

Building the GUI into a single application

To create a professional, user-friendly desktop application, the decision was made to package the Python GUI and its dependencies into a single

executable. This ensures that the police can run the system without manually setting up Python environments, installing dependencies, or dealing with conflicting library versions. The process required addressing several technical challenges:

- **Dependency Management**

Python packages such as customtkinter, PyAV, and Pillow rely on both Python code and underlying native libraries. To guarantee correct operation across systems, all dependencies were bundled using PyInstaller. This allows the GUI, decoding libraries, and associated plugins (for image/video processing) to be included in a single executable, avoiding runtime import errors.

- **Cross-Platform Considerations**

Although Linux is the primary development environment, care was taken to maintain compatibility with Windows. This included handling differences in file path separators, dynamic library loading, and ADB driver detection. Using a single-file approach ensures that users do not need to manually configure environment variables or install additional libraries.

- **Real-Time Performance**

Packaging into a single executable does not compromise the responsiveness of the application. By keeping separate threads for receiving and decoding the video stream, updating the GUI, and forwarding touch input, the system maintains low-latency interaction comparable to running the code directly from Python.

- **User Experience Enhancements**

Beyond technical packaging, the single-application approach allows the GUI to present a polished interface: a consistent window layout, embedded media previews, and interactive controls for device touch and media selection. Investigators can operate the system entirely via mouse and keyboard, without requiring terminal commands or manual ADB commands.

Reasoning Against Docker

While Docker was initially considered as a packaging option, several limitations made it unsuitable for this project (*Can I Run GUI Apps in Docker?*, 2024):

- **USB ADB Access:** Docker containers typically do not have direct access to USB devices without additional configuration. Forwarding ADB over USB through Docker adds complexity and often introduces latency or connection instability. Moreover, while this is easy enough to achieve on

a Linux based image that will run on a Linux OS, this is not as reliable for Windows.

- **GUI Limitations:** Running a GUI inside a Docker container requires X11 forwarding on Linux or virtual display servers, which is difficult to configure for cross-platform use. On Windows, GUI forwarding through Docker is even more challenging, often requiring third-party tools or WSL2.
- **Performance Concerns:** Docker introduces a lightweight virtualization layer. For real-time video decoding and touch input, even minimal overhead can cause noticeable lag or frame drops.
- **User Accessibility:** Investigators are expected to run the application with minimal technical setup. Requiring Docker installation, knowledge of container commands, and mounting of volumes would hinder usability and increase potential errors.

By packaging the Python application into a single executable, all dependencies, libraries, and resources are included, and the system remains fully operational without relying on an external container. This approach maximizes usability, reliability, and performance, particularly for environments where USB access and low-latency streaming are critical.

This solution as of yet though, only offers a built for Linux based OS's. This is due to Pyinstaller's nature, it builds an executable for the OS that is building the application. Based on research, in order to build the executable for Windows, will require building it in a Windows environment.

This is where Docker will come to play. A Docker container running a Windows image will be used for the building of the image.

(*Microsoft/Windows - Docker Image*, n.d.) After that is confirmed to work, a pipeline will be set up that builds both containers upon pushing the codebase to the Master branch of the Github repository.

Integration With the Virtual Camera Module

The desktop system integrates with the Android virtual camera module indirectly. The module intercepts the camera pipeline and replaces the camera output, while the desktop system:

- Manages media sent to the device, which will substitute the camera's stream
- Streams the device screen to make all operations visible in one window (folders on computer, media to upload to device, direct view of result on

phone)

- Enables touch interaction to operate Snapchat remotely

This layered approach preserves modularity: changes to the desktop interface do not affect the Android camera interception logic, and vice versa.

Testing and Validation

Multiple tests were done to ensure the reliability of the stream and touch responsiveness:

- Testing of stream decoding and frame rendering, both at the start of the stream, as well as after a few moments of inactivity (which was one of the downsides of the previous attempts, the stream would no longer receive new frames and crash)
- Testing using Snapchat to verify absence of the media upload tag
- Testing of touch (tap, swipe) using 3rd party apps on phone, such as 'Touch Screen Test' (*Touch Screen Test - Apps on Google Play*, n.d.)

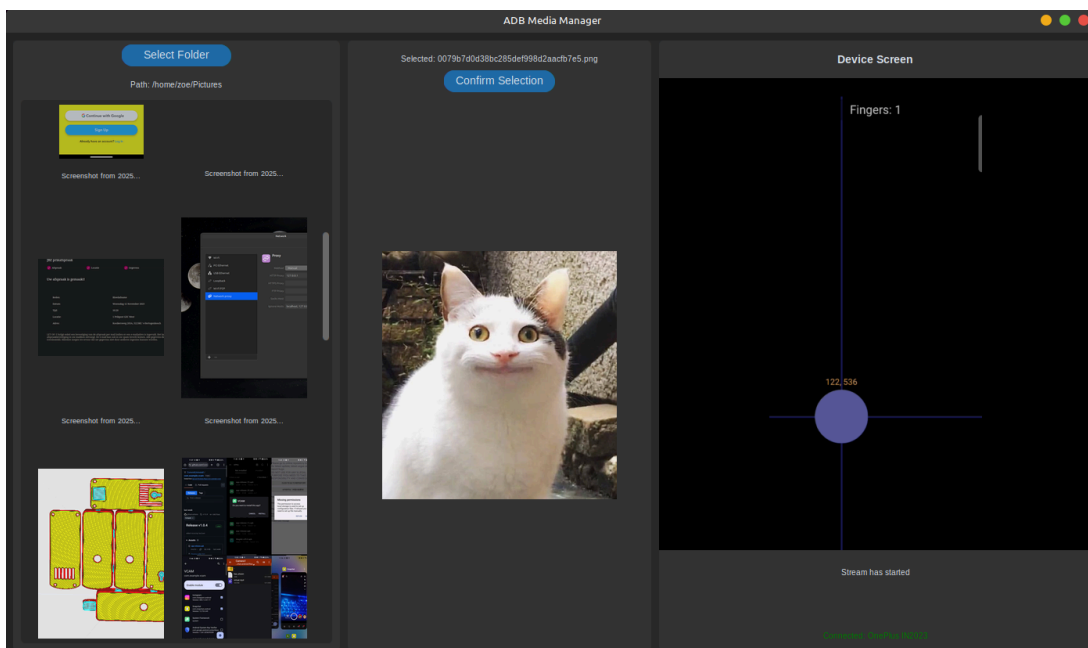


image-108.png

Testing was performed after each significant change to ensure stream stability and input accuracy.

Mermaid Diagram of GUI (Simplified)

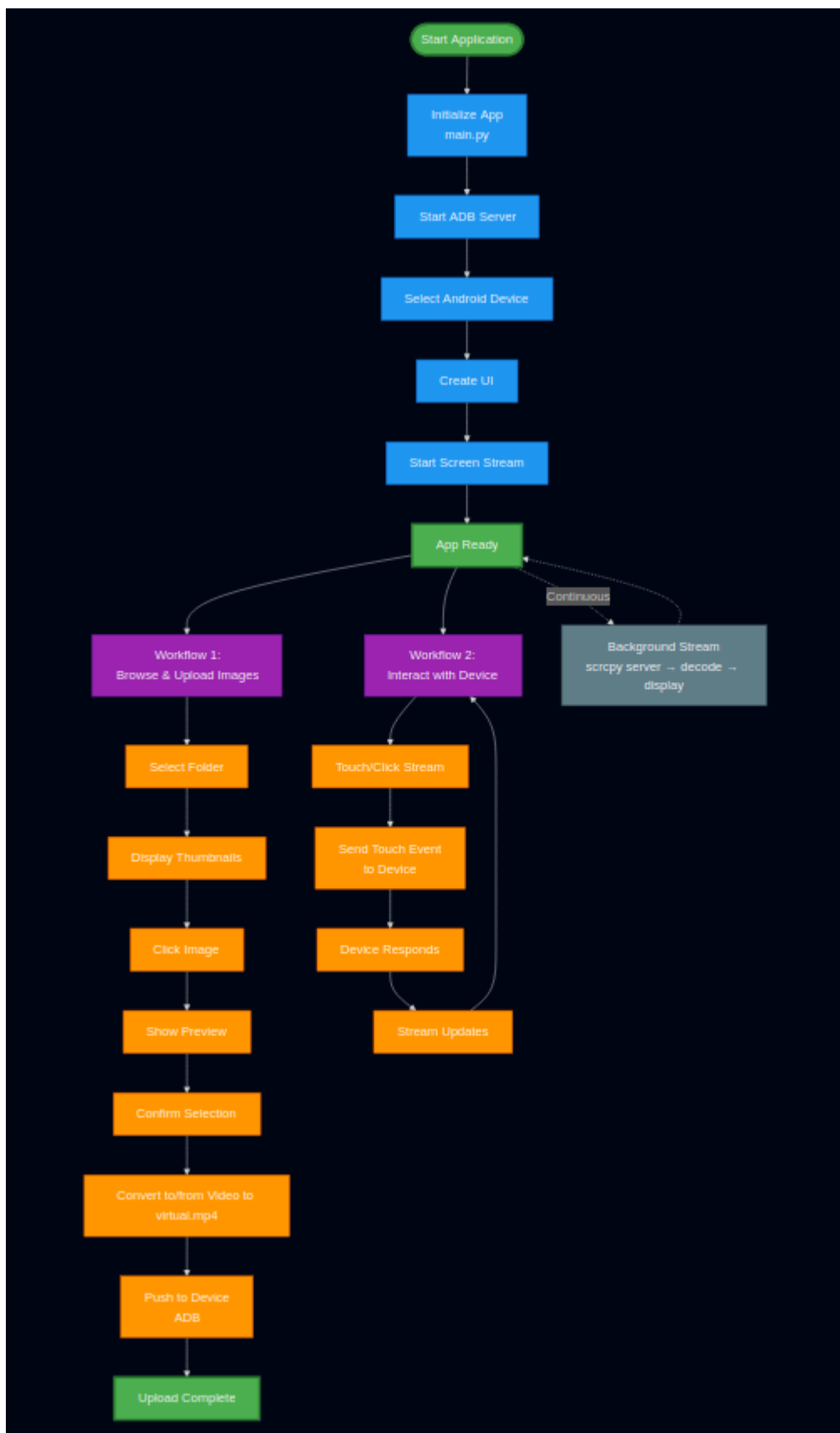


image-110.png

Next Steps

While the current solution significantly improves usability and is ready to use in a Linux environment, further steps are required:

- Packaging the GUI for a Windows installation
- Creating a CI pipeline that automatically builds executables for both Linux and Windows
- Conducting usability testing with stakeholder, feedback

Risks

- **Cross-Platform Compatibility Risks**

Differences in ADB behavior and driver availability between Windows and Linux may introduce instability.

Mitigation: Each released version should be tested on both platforms prior to official release.

Mitigation 2: New classes should be written in an abstract way or in single-use functions that do not break the rest of the code.

- **Dependency Maintenance Risks**

Reliance on scrcpy, PyAV, and FFmpeg introduces upstream dependency risks.

Mitigation: Keep a list of working releases that can be easily rolled back when needed. Moreover, changes to the GUI's backend should be tested in virtual environments prior to official release.

Sources

Jason.Chuang. (2016, August 8). *Use adb screenrecord command to mirror Android screen to PC via USB*. Stack Overflow.

<https://stackoverflow.com/questions/31472962/use-adb-screenrecord-command-to-mirror-android-screen-to-pc-via-usb>

User. (2012, January 26). *adb forward not working*. Stack Overflow.

<https://stackoverflow.com/questions/8846942/adb-forward-not-working>

GeeksforGeeks. (2025, October 3). *Multithreading in Python*.

GeeksforGeeks. <https://www.geeksforgeeks.org/python/multithreading-python-set-1/>

PyAV-Org. (n.d.). *GitHub - PyAV-Org/PyAV: Pythonic bindings for FFmpeg's libraries*. GitHub. <https://github.com/PyAV-Org/PyAV>

Voce. (2022, January 24). *What are H.264 NAL units?* VOCAL Technologies.

<https://vocal.com/video-codecs/itu-h-264/what-are-h-264-nal-units/>

OpenCV: OpenCV modules. (n.d.). <https://docs.opencv.org/4.x/>

Can I run GUI apps in Docker? (2024, February 11). Docker Community Forums. <https://forums.docker.com/t/can-i-run-gui-apps-in-docker/139789>

microsoft/windows - Docker Image. (n.d.).
<https://hub.docker.com/r/microsoft/windows>

Touch Screen Test - apps on Google Play. (n.d.).
<https://play.google.com/store/apps/details?id=com.microsys.TouchScreenTest&hl=en>